МИНИСТЕРСТВО ЦИФРОВЫХ ТЕХНОЛОГИЙ РЕСПУБЛИКИ УЗБЕКИСТАН
ТАШКЕНТСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ
имени Мухаммада ал-Хоразмий

Дисциплина: «Введение в программную инженерию»

ПРАКТИЧЕСКАЯ РАБОТА № 11

Тестирование программного обеспечения на уровне компонентов: тест-план и тест-кейсы

Вариант 19: разработка АРМ администратора салона красоты

Выполнил: **Тожикулов Асилбек**
Группа: **M320-24**
Вариант: **19**
Принял: _____

Ташкент, 2026

1. Тема практической работы

Тестирование программного обеспечения на уровне компонентов: тест-план и тест-кейсы.

Цель работы: закрепление знаний по теоретическим сведениям. Научиться планировать тестирование компонентов программного обеспечения, разрабатывать тест-кейсы методами «чёрного ящика» (эквивалентное разбиение, анализ граничных значений) и «белого ящика» (тестирование базисного пути), применять драйверы и заглушки, вести протокол тестирования и оформлять отчёты о дефектах.

Вариант 19: АРМ администратора салона красоты (BeautyDesk). Использованы требования FR-03…FR-07 и вариант использования UC-01 «Записать клиента» (ПР 5), диаграмма классов подсистемы «Журнал записи» (ПР 6) и требование к качеству QR-01 «0 двойных записей» (ПР 9).

2. Тестируемые компоненты и тест-план

2.1. Перечень тестируемых компонентов

Для тестирования выбрана подсистема **«Журнал записи»** – ядро системы, от которого зависит главная функция администратора. По диаграмме классов из ПР 6 реализованы на Python три компонента и интерфейсы их зависимостей:

Таблица 1 – Тестируемые компоненты подсистемы «Журнал записи»

Компонент	Ответственность	Зависимости (заменяются при тестировании)
K1. BookingRules (rules.py)	проверка бизнес-правил записи: не в прошлом, не дальше 60 дней, сетка 15 мин, длительность услуги 15–240 мин	нет; текущее время передаётся параметром
K2. AvailabilityChecker (availability.py)	проверка, свободен ли мастер: смена, выходной, перерыв, другие записи; поиск 3 ближайших свободных окон	IScheduleProvider (графики), IAppointmentRepository (записи)
K3. BookingService (service.py)	запись клиента (UC-01): вызывает K1 и K2, блокирует слот, сохраняет запись, отправляет событие для напоминания	IAppointmentRepository, IScheduleProvider, ISlotLock (Redis), INotifier (Celery), часы

2.2. Тест-план

Таблица 2 – Тест-план компонентов

Раздел	Содержание
Объекты тестирования	компоненты K1–K3 версии v0.1 (исходный код в приложении А)
Проверяемые требования	FR-03 (длительность по справочнику), FR-04 (запрет записи вне смены и в перерыв), FR-05 (запрет пересечений и 3 ближайших окна), FR-07 (отменённая запись не занимает время), FR-09 (событие для напоминания), BR-01 (запись не в прошлом и не дальше 60 дней), BR-02 (сетка 15 мин), QR-01 (нет двойных записей), UC-01 (клиент должен быть в базе). Правила BR-01, BR-02 уточнены с владельцем продукта

Раздел	Содержание
Не тестируется	интерфейс пользователя, реальные PostgreSQL, Redis и Celery (будут на интеграционном уровне), производительность
Подход	модульное тестирование каждого компонента в изоляции: «чёрный ящик» (эквивалентное разбиение и граничные значения) для К1 и К3, «белый ящик» (базисные пути) для метода K2.is_free, тесты взаимодействия К3 с заглушками; все тесты автоматизированы на pytest
Окружение и средства	Python 3, pytest 9.1, pytest-cov 7.1, VS Code; зависимости заменены заглушками: StubSchedule, FakeRepository, StubLock, SpyNotifier; фиксированное «текущее время» 16.11.2026 10:00 для воспроизводимости
Критерии начала	код компонентов собран без ошибок; интерфейсы зависимостей утверждены (ПР 6); тест-кейсы прошли ревью
Критерии завершения	все тест-кейсы выполнены, 100 % пройдено после исправлений; нет открытых дефектов Critical/Major; покрытие кода компонентов не ниже 70 % (NFR-06); протокол и отчёты о дефектах оформлены
Критерии приостановки	более 30 % тестов падают из-за одной причины (например, ошибка в общей заглушке) – прогон останавливается до исправления
Роли	Шахзод Каримов – тест-аналитик (тест-кейсы, протокол, отчёты о дефектах); Бахтиёр Юсупов – разработчик (код компонентов, исправление дефектов); Асилбек Тожикулов – Scrum-мастер (ревью тест-плана, контроль критериев завершения)
Сроки	день 1 – тест-план и тест-кейсы; день 2 – автотесты и первый прогон; день 3 – исправление дефектов и повторный прогон
Риски	заглушки ведут себя не так, как реальные PostgreSQL/Redis (снижение: те же сценарии повторить на интеграционном уровне); ошибки с часовым поясом и датами (снижение: фиксированное время в тестах); пропуск граничных случаев (снижение: анализ граничных значений по каждому условию)

3. Тестирование методом «чёрного ящика»

Тестируются бизнес-правила записи в К1 и К3. Тестовые данные: текущее время – понедельник 16.11.2026 10:00; смена мастера М1 – 09:00–21:00, воскресенье – выходной, перерыв 14:00–15:00; 17.11 у М1 есть запись 11:00–12:00 и отменённая запись 16:00–17:00; услуга S60 – 60 минут.

Таблица 3 – Классы эквивалентности и граничные значения

Входное условие	Допустимые классы	Недопустимые классы	Граничные значения
У1. Время начала относительно текущего	Д1: от «сейчас» до +60 дней включительно	Н1: в прошлом; Н2: дальше +60 дней	16.11 10:00 (= сейчас) / 09:45; 15.01.2027 (+60) / 16.01.2027 (+61)
У2. Минуты начала	Д2: кратны 15 (00, 15, 30, 45)	Н3: не кратны 15	10:45 / 10:10
У3. Длительность услуги, мин	Д3: 15–240	Н4: < 15; Н5: > 240	15, 240 / 14, 241

Входное условие	Допустимые классы	Недопустимые классы	Граничные значения
У4. Положение относительно смены	Д4: начало и конец внутри смены 09:00–21:00	Н6: начало до смены; Н7: конец после смены; Н8: выходной день	начало 09:00 / 08:45; конец 21:00 / 21:15
У5. Пересечение с занятым временем	Д5: не пересекается (в том числе встык и с отменённой записью)	Н9: пересекается с записью; Н10: заходит на перерыв	начало 12:00 после записи до 12:00 (встык) / 11:45
У6. Клиент	Д6: есть в базе	Н11: нет в базе	–

Таблица 4 – Тест-кейсы «чёрного ящика»

ID	Класс / граница	Входные данные	Ожидаемый результат	Прогон 1
ВВ01	Д1	17.11 10:00, 60 мин	правила выполнены	пройден
ВВ02	Д1, граница	16.11 10:00 (= сейчас)	правила выполнены	пройден
ВВ03	Н1, граница	16.11 09:45	ошибка PAST	пройден
ВВ04	Д1, граница	15.01.2027 10:00 (+60 дней)	правила выполнены	не пройден (DEF-01)
ВВ05	Н2, граница	16.01.2027 10:00 (+61 день)	ошибка TOO_FAR	пройден
ВВ06	Д2	17.11 10:45	правила выполнены	пройден
ВВ07	Н3	17.11 10:10	ошибка GRID	пройден
ВВ08	Д3, граница	длительность 15	правила выполнены	пройден
ВВ09	Д3, граница	длительность 240	правила выполнены	пройден
ВВ10	Н4, граница	длительность 14	ошибка DURATION	пройден
ВВ11	Н5, граница	длительность 241	ошибка DURATION	пройден
ВВ12	Д4, граница	С1, М1, S60, 17.11 09:00	запись 09:00–10:00 создана	пройден
ВВ13	Н6, граница	17.11 08:45	ошибка NOT_FREE	пройден
ВВ14	Д4, граница	17.11 20:00 (конец 21:00)	запись создана	пройден
ВВ15	Н7, граница	17.11 20:15 (конец 21:15)	ошибка NOT_FREE	пройден
ВВ16	Н8	22.11 (воскресенье) 10:00	ошибка NOT_FREE	пройден
ВВ17	Д5, граница	17.11 12:00 – встык с 11:00–12:00	запись 12:00–13:00 создана	не пройден (DEF-02)
ВВ18	Н9, граница	17.11 11:45	ошибка NOT_FREE	пройден
ВВ19	Н10	17.11 13:30 (до 14:30)	ошибка NOT_FREE	пройден
ВВ20	Д5	17.11 16:00 – время отменённой записи	запись создана	пройден
ВВ21	Н11	клиент С9	ошибка NO_CLIENT	пройден

Каждый допустимый класс покрыт хотя бы одним тестом, каждый недопустимый – отдельным тестом, чтобы одна ошибка не маскировала другую. Тесты ВВ01–ВВ11 проверяют К1 напрямую, ВВ12–ВВ21 – через К3 вместе с К2.

4. Тестирование методом «белого ящика»

Для анализа выбран метод **AvailabilityChecker.is_free(master_id, slot)** – в нём пять условий: выходной день, начало до смены, конец после смены, цикл по занятым интервалам и проверка пересечения. Номера узлов потокового графа указаны в комментариях кода (листинг 1).

Листинг 1 – Компонент AvailabilityChecker (availability.py), версия v0.2 после исправления DEF-02

```
"""Компонент 2. AvailabilityChecker – проверка свободного времени мастера (FR-04, FR-05, FR-
07)."""
from datetime import timedelta
from .models import TimeSlot, Status


class AvailabilityChecker:
    def __init__(self, schedule, repo):
        self.schedule = schedule
        self.repo = repo

    def is_free(self, master_id: str, slot: TimeSlot) -> bool:
        shift = self.schedule.shift_of(master_id, slot.start.date())   # 1
        if shift is None:                                               # 2
            return False                                                # 3
        if slot.start < shift.start:                                    # 4
            return False                                                # 5
        if slot.end > shift.end:                                        # 6
            return False                                                # 7
        for busy in self._busy_slots(master_id, slot.start.date()):     # 8
            if self._overlaps(slot, busy):                              # 9
                return False                                            # 10
        return True                                                     # 11

    def nearest_slots(self, master_id: str, start, minutes: int, n: int = 3, step: int = 15):
        """Ближайшие n свободных окон в тот же день после желаемого времени (FR-05)."""
        result, t = [], start + timedelta(minutes=step)
        shift = self.schedule.shift_of(master_id, start.date())
        while shift and t + timedelta(minutes=minutes) <= shift.end and len(result) < n:
            cand = TimeSlot(t, t + timedelta(minutes=minutes))
            if self.is_free(master_id, cand):
                result.append(cand)
            t += timedelta(minutes=step)
        return result

    def _busy_slots(self, master_id, day):
        busy = list(self.schedule.breaks_of(master_id, day))
        busy += [a.slot for a in self.repo.find_by_master_and_day(master_id, day)
                 if a.status not in (Status.CANCELLED, Status.NO_SHOW)]
        return busy

    @staticmethod
    def _overlaps(a: TimeSlot, b: TimeSlot) -> bool:
        return a.start < b.end and b.start < a.end
```

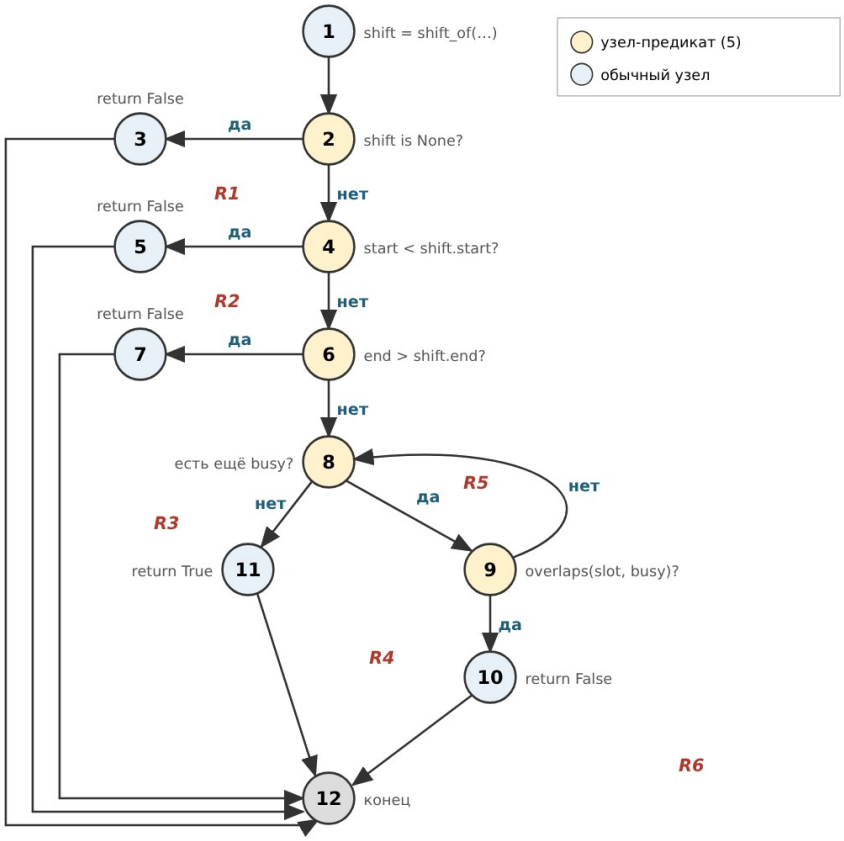


Рисунок 1 – Потоковый граф метода is_free (построен в draw.io)

Цикломатическая сложность вычислена тремя способами:

1) по рёбрам и узлам: V(G) = E − N + 2 = 16 − 12 + 2 = **6**;
2) по узлам-предикатам: V(G) = P + 1 = 5 + 1 = **6** (предикаты – узлы 2, 4, 6, 8, 9);
3) по числу регионов графа: R1–R5 внутренние и R6 внешний, V(G) = **6**.

Все три способа дали одинаковый результат, значит, базисное множество состоит из 6 независимых путей. Каждый следующий путь добавляет хотя бы одно новое ребро.

Таблица 5 – Базисные пути и тест-кейсы «белого ящика»

ID	Базисный путь	Входные данные (мастер, интервал)	Ожидается	Прогон 1
WB01	1-2-3-12	M1, вс 22.11 10:00–11:00 (выходной)	False	пройден
WB02	1-2-4-5-12	M1, 18.11 08:30–09:30	False	пройден
WB03	1-2-4-6-7-12	M1, 18.11 20:30–21:30	False	пройден
WB04	1-2-4-6-8-11-12	M2 (нет перерывов и записей), 18.11 10:00–11:00	True	пройден
WB05	1-2-4-6-8-9-10-12	M1, 18.11 14:30–15:30 (первый занятый – перерыв)	False	пройден
WB06	1-2-4-6-8-9-8-11-12	M1, 18.11 10:00–11:00 (перерыв есть, не пересекается)	True	пройден

Все тесты «белого ящика» прошли и в первом прогоне, хотя в методе _overlaps был дефект DEF-02. Покрытие всех путей не гарантирует проверку граничных значений внутри условий: дефект проявляется только при интервалах «встык», и его нашёл тест «чёрного ящика» BB17. Поэтому два метода дополняют друг друга.

5. Драйверы, заглушки, тесты и протоколы прогонов

5.1. Драйверы и заглушки

Заглушки заменяют зависимости компонентов: StubSchedule возвращает фиксированные смены и перерывы, FakeRepository хранит записи в памяти вместо PostgreSQL, StubLock имитирует блокировку слота в Redis (в том числе занятую другим администратором), SpyNotifier запоминает отправленные события вместо Celery и Telegram. **Драйверы** – фикстуры pytest в conftest.py, которые собирают компонент с заглушками и передают его тестам, а также ручной драйвер driver_manual.py для отладки.

Листинг 2 – Заглушки (tests/stubs.py)

```
"""Заглушки (stubs), фиктивные объекты (fakes) и шпионы (spies) для зависимостей компонентов."""
from datetime import datetime, time
from booking.interfaces import IScheduleProvider, IAppointmentRepository, ISlotLock, INotifier
from booking.models import TimeSlot


def dt(day: str, hhmm: str) -> datetime:
    return datetime.fromisoformat(f"{day}T{hhmm}")


class StubSchedule(IScheduleProvider):
    """Смена 09:00–21:00 каждый день, воскресенье – выходной; у M1 перерыв 14:00–15:00."""
    DURATIONS = {"S15": 15, "S60": 60, "S240": 240, "S14": 14, "S241": 241}

    def shift_of(self, master_id, day):
        if day.weekday() == 6:
            return None
        return TimeSlot(datetime.combine(day, time(9, 0)), datetime.combine(day, time(21, 0)))

    def breaks_of(self, master_id, day):
        if master_id == "M1":
            return [TimeSlot(datetime.combine(day, time(14, 0)), datetime.combine(day, time(15,
0)))]
        return []

    def duration_of(self, service_id):
        return self.DURATIONS[service_id]


class FakeRepository(IAppointmentRepository):
    """Хранилище в памяти вместо PostgreSQL."""
    def __init__(self, clients=("C1",)):
        self.items, self.clients = [], set(clients)

    def save(self, appointment):
        self.items.append(appointment)

    def find_by_master_and_day(self, master_id, day):
        return [a for a in self.items if a.master_id == master_id and a.slot.start.date() == day]
```

```
    def client_exists(self, client_id):
        return client_id in self.clients


class StubLock(ISlotLock):
    """Заглушка блокировки слота (в системе – Redis). free=False имитирует занятый слот."""
    def __init__(self, free=True):
        self.free, self.acquired, self.released = free, 0, 0

    def acquire(self, master_id, slot):
        self.acquired += 1
        return self.free

    def release(self, master_id, slot):
        self.released += 1


class SpyNotifier(INotifier):
    """Шпион: запоминает, о каких записях было отправлено событие (вместо Celery/Telegram)."""
    def __init__(self):
        self.sent = []

    def appointment_created(self, appointment):
        self.sent.append(appointment)
```

Листинг 3 – Драйверы-фикстуры (tests/conftest.py)

```
"""Драйверы: фикстуры pytest собирают тестируемые компоненты с заглушками."""
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))
import pytest
from booking.models import Appointment, Status
from booking.rules import BookingRules
from booking.availability import AvailabilityChecker
from booking.service import BookingService
from tests.stubs import StubSchedule, FakeRepository, StubLock, SpyNotifier, dt
from booking.models import TimeSlot

NOW = dt("2026-11-16", "10:00")          # понедельник, «текущее время» теста


@pytest.fixture
def schedule():
    return StubSchedule()


@pytest.fixture
def repo():
    r = FakeRepository(clients=("C1", "C2"))
    # 17.11: у M1 запись 11:00–12:00 и отменённая запись 16:00–17:00
    r.save(Appointment("C2", "M1", "S60", TimeSlot(dt("2026-11-17", "11:00"), dt("2026-11-17",
"12:00"))))
    r.save(Appointment("C2", "M1", "S60", TimeSlot(dt("2026-11-17", "16:00"), dt("2026-11-17",
"17:00")),
                       status=Status.CANCELLED))
    return r


@pytest.fixture
def rules():
    return BookingRules()
```

```
@pytest.fixture
def checker(schedule, repo):
    return AvailabilityChecker(schedule, repo)


@pytest.fixture
def lock():
    return StubLock()


@pytest.fixture
def notifier():
    return SpyNotifier()


@pytest.fixture
def service(rules, checker, schedule, repo, lock, notifier):
    return BookingService(rules, checker, schedule, repo, lock, notifier, clock=lambda: NOW)
```

Листинг 4 – Ручной драйвер (driver_manual.py)

```
"""Ручной драйвер: вызывает BookingService с заглушками и печатает результат (для отладки)."""
from booking.rules import BookingRules
from booking.availability import AvailabilityChecker
from booking.service import BookingService
from booking.models import BookingError
from tests.stubs import StubSchedule, FakeRepository, StubLock, SpyNotifier, dt

schedule, repo = StubSchedule(), FakeRepository(clients=("C1",))
svc = BookingService(BookingRules(), AvailabilityChecker(schedule, repo), schedule, repo,
                     StubLock(), SpyNotifier(), clock=lambda: dt("2026-11-16", "10:00"))
for start in ["2026-11-17T11:00", "2026-11-17T11:30", "2026-11-17T12:00"]:
    try:
        a = svc.book("C1", "M1", "S60", dt(*start.split("T")))
        print(f"{start}: записан, {a.slot.start:%H:%M}-{a.slot.end:%H:%M}, id={a.id}")
    except BookingError as e:
        print(f"{start}: {e.code} – {e}; варианты: {[f'{s.start:%H:%M}' for s in e.suggestions]}")
```

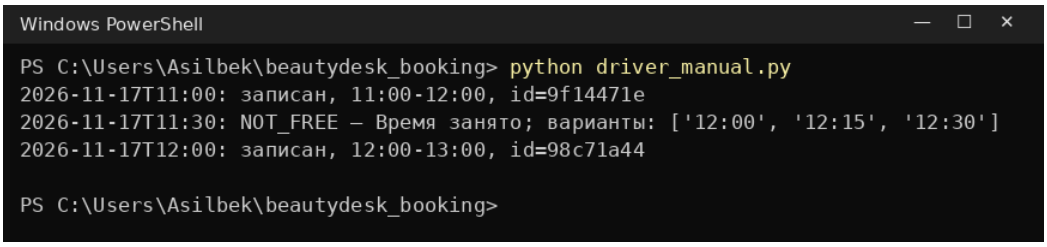


Рисунок 2 – Запуск ручного драйвера после исправлений

5.2. Автоматизированные тесты

Листинг 5 – Тесты «чёрного ящика» (tests/test_blackbox.py)

```
"""Тест-кейсы «чёрного ящика»: эквивалентное разбиение и граничные значения (раздел 3 отчёта)."""
import pytest
from booking.models import BookingError
from tests.stubs import dt
from tests.conftest import NOW


def code_of(fn):
    with pytest.raises(BookingError) as e:
        fn()
    return e.value.code
```

```
# --- У1, У2, У3: правила записи (компонент BookingRules) ---
RULES = [
    ("BB01", dt("2026-11-17", "10:00"), 60, None),        # К1.1 будущее
    ("BB02", dt("2026-11-16", "10:00"), 60, None),        # граница: ровно «сейчас»
    ("BB03", dt("2026-11-16", "09:45"), 60, "PAST"),      # граница: на 15 мин раньше
    ("BB04", dt("2027-01-15", "10:00"), 60, None),        # граница: +60 дней
    ("BB05", dt("2027-01-16", "10:00"), 60, "TOO_FAR"),   # граница: +61 день
    ("BB06", dt("2026-11-17", "10:45"), 60, None),        # минуты кратны 15
    ("BB07", dt("2026-11-17", "10:10"), 60, "GRID"),      # минуты не кратны 15
    ("BB08", dt("2026-11-17", "10:00"), 15, None),        # граница: 15 мин
    ("BB09", dt("2026-11-17", "10:00"), 240, None),       # граница: 240 мин
    ("BB10", dt("2026-11-17", "10:00"), 14, "DURATION"),  # граница: 14 мин
    ("BB11", dt("2026-11-17", "10:00"), 241, "DURATION"), # граница: 241 мин
]


@pytest.mark.parametrize("tc, start, duration, expected", RULES, ids=[r[0] for r in RULES])
def test_rules(rules, tc, start, duration, expected):
    if expected is None:
        rules.check(start, duration, NOW)
    else:
        assert code_of(lambda: rules.check(start, duration, NOW)) == expected


# --- У4, У5, У6: запись через BookingService (смена, перерыв, пересечения, клиент) ---
BOOKING = [
    ("BB12", "C1", dt("2026-11-17", "09:00"), None),        # граница: начало смены
    ("BB13", "C1", dt("2026-11-17", "08:45"), "NOT_FREE"),  # до начала смены
    ("BB14", "C1", dt("2026-11-17", "20:00"), None),        # граница: конец ровно 21:00
    ("BB15", "C1", dt("2026-11-17", "20:15"), "NOT_FREE"),  # конец 21:15 – после смены
    ("BB16", "C1", dt("2026-11-22", "10:00"), "NOT_FREE"),  # воскресенье – выходной
    ("BB17", "C1", dt("2026-11-17", "12:00"), None),        # граница: встык с записью 11–12
    ("BB18", "C1", dt("2026-11-17", "11:45"), "NOT_FREE"),  # пересечение на 15 мин
    ("BB19", "C1", dt("2026-11-17", "13:30"), "NOT_FREE"),  # заходит на перерыв 14–15
    ("BB20", "C1", dt("2026-11-17", "16:00"), None),        # отменённая запись не блокирует
    ("BB21", "C9", dt("2026-11-17", "10:00"), "NO_CLIENT"), # клиента нет в базе
]


@pytest.mark.parametrize("tc, client, start, expected", BOOKING, ids=[r[0] for r in BOOKING])
def test_booking(service, tc, client, start, expected):
    if expected is None:
        a = service.book(client, "M1", "S60", start)
        assert a.slot.start == start and a.slot.minutes == 60
    else:
        assert code_of(lambda: service.book(client, "M1", "S60", start)) == expected
```

Листинг 6 – Тесты «белого ящика» (tests/test_whitebox.py)

```
"""Тест-кейсы «белого ящика»: базисные пути метода AvailabilityChecker.is_free (раздел 4)."""
from booking.models import TimeSlot
from tests.stubs import dt


def slot(day, a, b):
    return TimeSlot(dt(day, a), dt(day, b))


def test_wb01_path1_day_off(checker):            # 1-2-3-12
    assert checker.is_free("M1", slot("2026-11-22", "10:00", "11:00")) is False
```

```
def test_wb02_path2_before_shift(checker):       # 1-2-4-5-12
    assert checker.is_free("M1", slot("2026-11-18", "08:30", "09:30")) is False


def test_wb03_path3_after_shift(checker):        # 1-2-4-6-7-12
    assert checker.is_free("M1", slot("2026-11-18", "20:30", "21:30")) is False


def test_wb04_path4_no_busy(checker):            # 1-2-4-6-8-11-12
    assert checker.is_free("M2", slot("2026-11-18", "10:00", "11:00")) is True


def test_wb05_path5_overlap_first(checker):      # 1-2-4-6-8-9-10-12
    assert checker.is_free("M1", slot("2026-11-18", "14:30", "15:30")) is False


def test_wb06_path6_busy_no_overlap(checker):    # 1-2-4-6-8-9-8-11-12
    assert checker.is_free("M1", slot("2026-11-18", "10:00", "11:00")) is True
```

Листинг 7 – Тесты взаимодействия с заглушками (tests/test_integration.py)

```
"""Тесты взаимодействия BookingService с заглушками зависимостей."""
import pytest
from booking.models import BookingError
from tests.stubs import dt


def test_it01_success_saved_notified_unlocked(service, repo, lock, notifier):
    a = service.book("C1", "M1", "S60", dt("2026-11-17", "09:00"))
    assert a in repo.items                       # запись сохранена
    assert notifier.sent == [a]                  # событие для напоминания отправлено 1 раз
    assert lock.acquired == lock.released == 1   # блокировка снята


def test_it02_slot_locked_by_other_admin(service, repo, lock, notifier):
    lock.free = False                            # другой администратор оформляет то же время
    before = len(repo.items)
    with pytest.raises(BookingError) as e:
        service.book("C1", "M1", "S60", dt("2026-11-17", "09:00"))
    assert e.value.code == "LOCKED"
    assert len(repo.items) == before and notifier.sent == []


def test_it03_not_free_suggests_three_nearest(service, lock):
    with pytest.raises(BookingError) as e:
        service.book("C1", "M1", "S60", dt("2026-11-17", "11:45"))
    starts = [s.start.strftime("%H:%M") for s in e.value.suggestions]
    assert starts == ["12:00", "12:15", "12:30"]
    assert lock.released == 1                    # блокировка снята и при ошибке
```

Таблица 6 – Тест-кейсы взаимодействия КЗ с зависимостями

ID	Сценарий	Проверки
IT01	успешная запись С1 к М1 на 17.11 09:00	запись сохранена в репозитории; событие отправлено ровно один раз; блокировка взята и снята
IT02	слот одновременно оформляет другой администратор	ошибка LOCKED; запись не сохранена; событие не отправлено (QR-01)
IT03	время 11:45 занято	ошибка NOT_FREE; предложены 12:00, 12:15, 12:30 (FR-05); блокировка снята и при ошибке

5.3. Протоколы прогонов

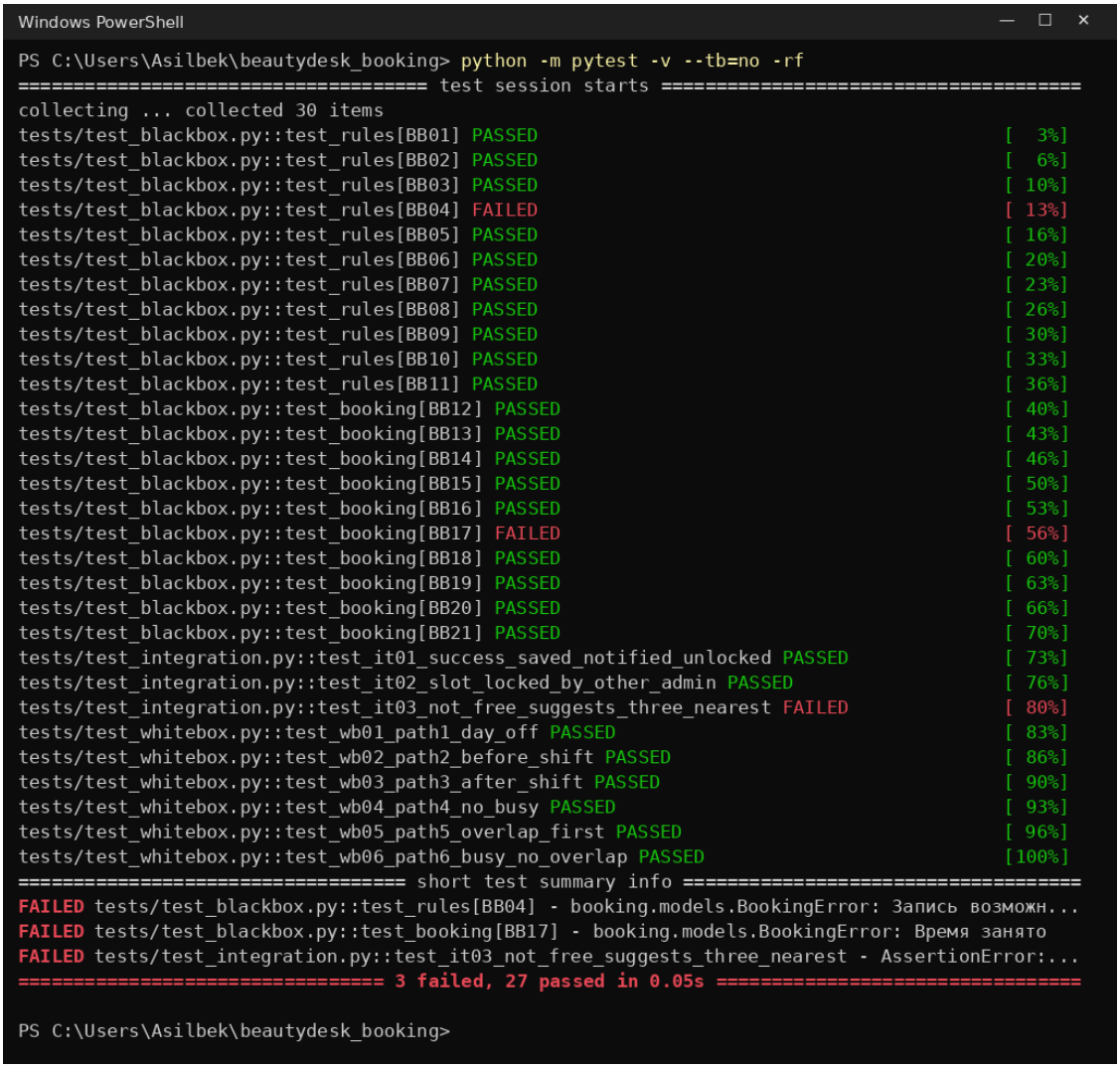


Рисунок 3 – Прогон 1 – исходная версия v0.1: 27 пройдено, 3 не пройдено

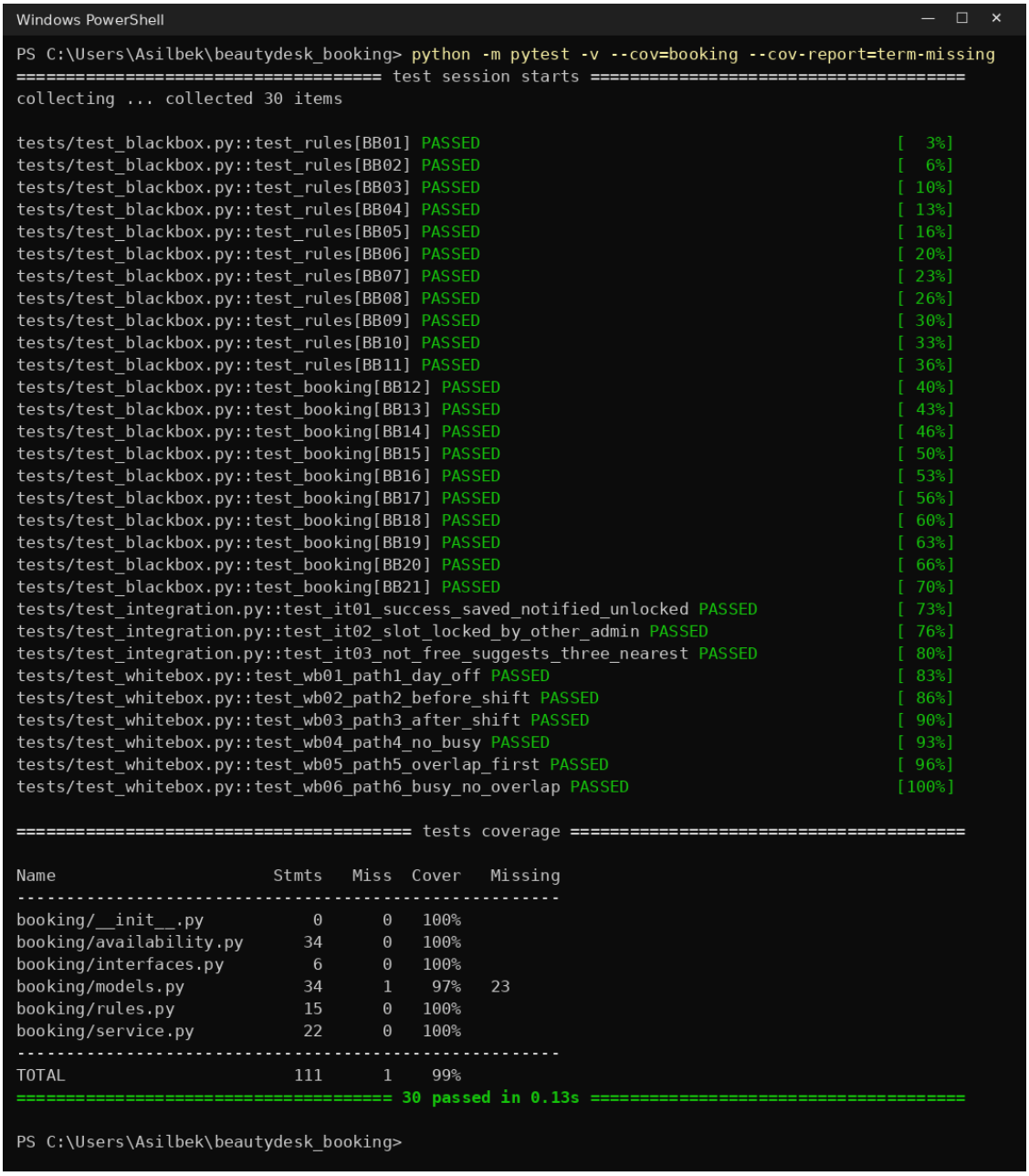


Рисунок 4 – Прогон 2 – после исправления DEF-01 и DEF-02: 30 из 30 пройдено, покрытие 99 %

Таблица 7 – Сводный протокол тестирования

Прогон	Дата	Версия кода	Всего	Пройдено	Не пройдено	Не пройдены
1	11.11.2026	v0.1 – исходная	30	27	3	BB04, BB17, IT03
2	11.11.2026	v0.2 – исправлены DEF-01, DEF-02	30	30	0	–

Во втором прогоне повторно выполнены **все** 30 тестов, а не только упавшие, чтобы убедиться, что исправления ничего не сломали (регрессионное тестирование). Покрытие операторов – 99 %; не покрыта одна строка в models.py – выброс исключения в конструкторе TimeSlot при конце интервала раньше начала, потому что

сервис всегда строит интервал из положительной длительности. Требование NFR-06 (покрытие модулей записи ≥ 70 %) выполнено.

6. Отчёты о дефектах и матрица трассируемости

6.1. Отчёты о дефектах

Таблица 8 – Отчёт о дефекте DEF-01

Поле	Значение
ID / название	DEF-01. Отклоняется запись ровно на 60 дней вперёд
Компонент, версия	K1 BookingRules.check, rules.py, строка 14; v0.1
Обнаружен	тест BB04, прогон 1, 11.11.2026, Шахзод Каримов
Серьёзность / приоритет	Major / средний – клиенту нельзя записаться на последний разрешённый день, есть обходной путь (записать на день раньше)
Шаги воспроизведения	1) текущее время 16.11.2026 10:00; 2) вызвать check(start = 15.01.2027 10:00, duration = 60)
Ожидаемый результат	правило выполнено: по BR-01 запись допускается не дальше чем на 60 дней включительно
Фактический результат	BookingError TOO_FAR «Запись возможна не дальше чем на 60 дней»
Причина	в условии использовано days >= HORIZON_DAYS вместо days > HORIZON_DAYS (ошибка на единицу на границе)
Исправление	условие заменено на days > HORIZON_DAYS (v0.2), Бахтиёр Юсупов
Статус	Новый → Исправлен → Проверен (прогон 2: BB04, BB05 пройдены) → Закрыт

Таблица 9 – Отчёт о дефекте DEF-02

Поле	Значение
ID / название	DEF-02. Запись «встык» к предыдущей считается пересечением
Компонент, версия	K2 AvailabilityChecker._overlaps, availability.py, строка 43; v0.1
Обнаружен	тесты BB17 и IT03, прогон 1, 11.11.2026, Шахзод Каримов
Серьёзность / приоритет	Critical / высокий – администратор не может записать клиента сразу после другого, мастер простаивает; это обычная ситуация в каждом салоне
Шаги воспроизведения	1) у мастера M1 запись 17.11 11:00–12:00; 2) записать клиента C1 к M1 на 17.11 12:00, услуга 60 мин
Ожидаемый результат	запись 12:00–13:00 создана; при попытке на 11:45 предлагаются окна 12:00, 12:15, 12:30
Фактический результат	BookingError NOT_FREE «Время занято»; в IT03 предложены 12:15, 12:30, 12:45 – окно 12:00 потеряно
Причина	в проверке пересечения использованы нестрогие сравнения a.start <= b.end and b.start <= a.end, из-за чего интервалы, у которых конец одного равен началу другого, считаются пересекающимися
Исправление	сравнения заменены на строгие: a.start < b.end and b.start < a.end (v0.2)
Статус	Новый → Исправлен → Проверен (прогон 2: BB17, BB18, IT03, WB05, WB06 пройдены) → Закрыт

6.2. Матрица трассируемости «требование – тест-кейс»

Таблица 10 – Матрица трассируемости

Требование	Содержание	Тест-кейсы	Статус (прогон 2)
BR-01	запись не в прошлом и не дальше 60 дней	BB01–BB05	пройдены
BR-02	начало по сетке 15 минут	BB06, BB07	пройдены
FR-03	длительность записи по справочнику услуг	BB08–BB11, BB12, IT01	пройдены
FR-04	запрет записи вне смены и в перерыв	BB12–BB16, BB19, WB01–WB03, WB05	пройдены
FR-05	запрет пересечений и 3 ближайших окна	BB17, BB18, WB04, WB06, IT03	пройдены
FR-07	отменённая запись не занимает время	BB20	пройден
FR-09	событие для напоминания после записи	IT01, IT02	пройдены
QR-01	нет двойной записи при одновременной работе	IT02	пройден
UC-01	запись только существующего клиента	BB21, IT01	пройдены

Каждое проверяемое требование покрыто хотя бы одним тест-кейсом, и каждый тест-кейс связан с требованием. Полная проверка QR-01 (1 000 параллельных попыток записи) относится к интеграционному уровню с реальным Redis и в эту работу не входит.

7. Вывод

В работе спланировано и выполнено модульное тестирование трёх компонентов подсистемы «Журнал записи» системы BeautyDesk: BookingRules, AvailabilityChecker и BookingService. Составлен тест-план с объектами, требованиями, подходом, окружением, критериями, ролями, сроками и рисками.

Методом «чёрного ящика» по шести входным условиям выделено 6 допустимых и 11 недопустимых классов эквивалентности, разработан 21 тест-кейс с граничными значениями. Для метода is_free построен потоковый граф, цикломатическая сложность V(G) = 6 подтверждена тремя способами, для 6 базисных путей разработаны тест-кейсы. Вместе с тремя тестами взаимодействия получилось 30 автотестов на pytest с заглушками и драйверами.

Первый прогон выявил 2 дефекта – оба на границах: запись ровно на 60-й день и запись «встык». После исправления повторный прогон всех тестов прошёл полностью, покрытие кода – 99 %. Я убедился, что основная часть ошибок находится

именно на границах, а тесты базисных путей без граничных значений их могут пропустить.

Приложение А. Исходный код компонентов (версия v0.2)

Листинг А.1 – Модели (booking/models.py)

```
from dataclasses import dataclass, field
from datetime import datetime, timedelta
from enum import Enum
import uuid


class Status(Enum):
    PLANNED = "запланирована"
    CONFIRMED = "подтверждена"
    IN_PROGRESS = "в работе"
    DONE = "завершена"
    CANCELLED = "отменена клиентом"
    NO_SHOW = "неявка"


@dataclass(frozen=True)
class TimeSlot:
    start: datetime
    end: datetime

    def __post_init__(self):
        if self.end <= self.start:
            raise ValueError("конец интервала должен быть позже начала")

    @property
    def minutes(self) -> int:
        return int((self.end - self.start).total_seconds() // 60)


@dataclass
class Appointment:
    client_id: str
    master_id: str
    service_id: str
    slot: TimeSlot
    status: Status = Status.PLANNED
    id: str = field(default_factory=lambda: uuid.uuid4().hex[:8])


class BookingError(Exception):
    def __init__(self, code: str, message: str, suggestions=None):
        super().__init__(message)
        self.code = code
        self.suggestions = suggestions or []
```

Листинг А.2 – Интерфейсы зависимостей (booking/interfaces.py)

```
"""Интерфейсы зависимостей компонентов (по диаграмме классов ПР 6)."""
from abc import ABC, abstractmethod
from datetime import date


class IScheduleProvider(ABC):
    @abstractmethod
    def shift_of(self, master_id: str, day: date): ...      # TimeSlot или None (выходной)
    @abstractmethod
    def breaks_of(self, master_id: str, day: date) -> list: ...
    @abstractmethod
    def duration_of(self, service_id: str) -> int: ...       # минуты
```

```
class IAppointmentRepository(ABC):
    @abstractmethod
    def save(self, appointment): ...
    @abstractmethod
    def find_by_master_and_day(self, master_id: str, day: date) -> list: ...
    @abstractmethod
    def client_exists(self, client_id: str) -> bool: ...


class ISlotLock(ABC):
    @abstractmethod
    def acquire(self, master_id: str, slot) -> bool: ...
    @abstractmethod
    def release(self, master_id: str, slot) -> None: ...


class INotifier(ABC):
    @abstractmethod
    def appointment_created(self, appointment) -> None: ...
```

Листинг А.3 – Компонент BookingRules (booking/rules.py)

```
"""Компонент 1. BookingRules – бизнес-правила записи (FR-03, BR-01, BR-02)."""
from datetime import datetime
from .models import BookingError

HORIZON_DAYS = 60          # BR-01: запись не дальше чем на 60 дней вперёд
GRID_MINUTES = 15          # BR-02: начало записи – по сетке 15 минут
MIN_DURATION, MAX_DURATION = 15, 240   # FR-03: длительность услуги из справочника


class BookingRules:
    def check(self, start: datetime, duration: int, now: datetime) -> None:
        if start < now:
            raise BookingError("PAST", "Нельзя записать на прошедшее время")
        if (start.date() - now.date()).days > HORIZON_DAYS:
            raise BookingError("TOO_FAR", f"Запись возможна не дальше чем на {HORIZON_DAYS} дней")
        if start.minute % GRID_MINUTES != 0 or start.second != 0:
            raise BookingError("GRID", "Время начала должно быть кратно 15 минутам")
        if not (MIN_DURATION <= duration <= MAX_DURATION):
            raise BookingError("DURATION", "Длительность услуги вне допустимого диапазона 15–240
мин")
```

Листинг А.4 – Компонент BookingService (booking/service.py)

```
"""Компонент 3. BookingService – запись клиента (UC-01), использует компоненты 1 и 2."""
from datetime import timedelta
from .models import Appointment, BookingError, TimeSlot


class BookingService:
    def __init__(self, rules, checker, schedule, repo, lock, notifier, clock):
        self.rules, self.checker, self.schedule = rules, checker, schedule
        self.repo, self.lock, self.notifier, self.clock = repo, lock, notifier, clock

    def book(self, client_id, master_id, service_id, start) -> Appointment:
        if not self.repo.client_exists(client_id):
            raise BookingError("NO_CLIENT", "Клиент не найден")
        duration = self.schedule.duration_of(service_id)
        self.rules.check(start, duration, self.clock())
        slot = TimeSlot(start, start + timedelta(minutes=duration))
        if not self.lock.acquire(master_id, slot):
```

```
            raise BookingError("LOCKED", "Это время сейчас оформляет другой администратор")
        try:
            if not self.checker.is_free(master_id, slot):
                raise BookingError("NOT_FREE", "Время занято",
                                   self.checker.nearest_slots(master_id, start, duration))
            appointment = Appointment(client_id, master_id, service_id, slot)
            self.repo.save(appointment)
        finally:
            self.lock.release(master_id, slot)
        self.notifier.appointment_created(appointment)
        return appointment
```